EXPERIMENT NO. 1

Implementation of Fundamental Image Processing Operations using Python and OpenCV

Aim

To implement fundamental image processing techniques including image acquisition, storage, color-space conversion, geometric transformations, and image enhancement using Python and OpenCV.

Purpose

Image processing is a core part of computer vision applications such as autonomous driving, healthcare, surveillance, robotics, and industrial automation. This experiment explores how digital images can be read, inspected, converted, resized, transformed, and enhanced before being used in advanced computer vision tasks.

Prerequisites

- Basic Python programming
- Understanding of pixels, resolution, and color channels
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Jupyter Notebook, Google Colab, or Visual Studio Code

Software / Libraries

Python 3, OpenCV, NumPy, and Matplotlib.

Installation

```
pip install opencv-python numpy matplotlib
```

Note: Replace `image.jpg` with the path to an image available on your computer. The example outputs in this report use a simple illustrative image.

Program and Execution

Cell 1 — Import required libraries and load an image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

if image is None:
    raise FileNotFoundError("Image not found. Check the file path.")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

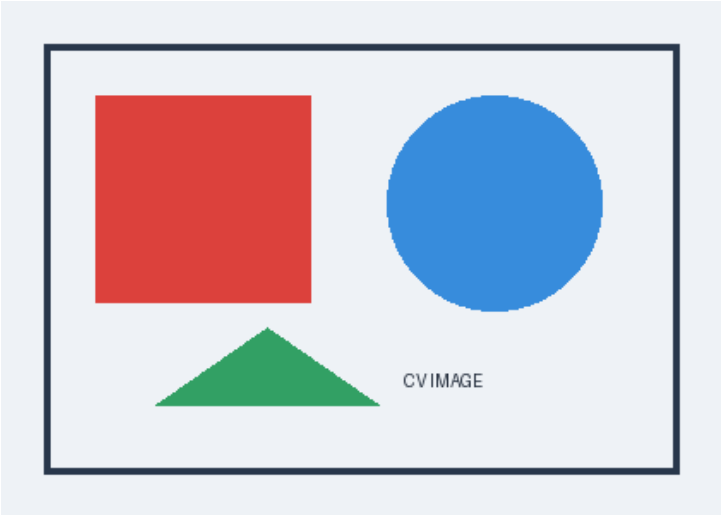
print("Image loaded successfully")
print("Dimensions (height, width, channels):", image.shape)
print("Data type:", image.dtype)
print("Total pixels:", image.shape[0] * image.shape[1])
```

```
Image loaded successfully
Dimensions (height, width, channels): (image_height, image_width, 3)
Data type: uint8
Total pixels: image_height * image_width
```

OpenCV reads color images in BGR order. Converting to RGB allows Matplotlib to display the colors correctly.

Cell 2 — Display the original image

```
plt.figure(figsize=(6, 4))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```



Cell 3 — Save the image in JPEG and PNG formats

```
cv2.imwrite("saved_image.jpg", image)
cv2.imwrite("saved_image.png", image)

print("Image saved in JPEG and PNG formats.")
```

```
Image saved in JPEG and PNG formats.
```

JPEG uses lossy compression and is often smaller. PNG uses lossless compression and is suitable when preserving pixel values is important.

Color-Space Conversion

Cell 4 — Convert the image to Grayscale, HSV, and LAB

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

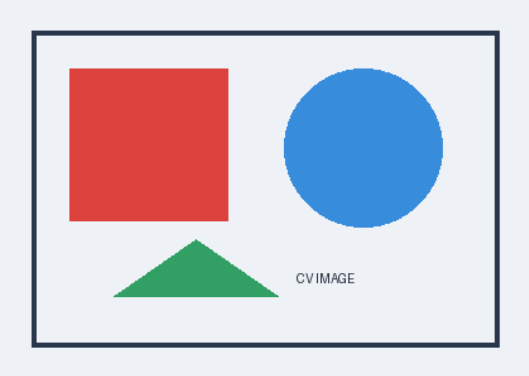
print("Original shape:", image.shape)
print("Grayscale shape:", gray.shape)
print("HSV shape:", hsv.shape)
print("LAB shape:", lab.shape)

plt.figure(figsize=(12, 4))
plt.subplot(1, 4, 1); plt.imshow(image_rgb); plt.title("Original"); plt.axis("off")
plt.subplot(1, 4, 2); plt.imshow(gray, cmap="gray"); plt.title("Grayscale"); plt.axis("off")
plt.subplot(1, 4, 3); plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)); plt.title("HSV"); plt.axis("off")
plt.subplot(1, 4, 4); plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)); plt.title("LAB"); plt.axis("off")
plt.tight_layout()
plt.show()
```

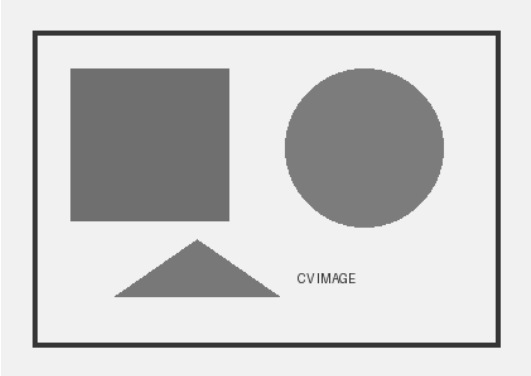
```
Original shape: (H, W, 3)
Grayscale shape: (H, W)
HSV shape: (H, W, 3)
LAB shape: (H, W, 3)
```

Grayscale stores intensity only. HSV separates hue, saturation, and value. LAB represents lightness separately from color-opponent information and can be useful for color-based analysis.

Illustration — Original and grayscale appearance



Original color image



Grayscale image

Geometric Transformations and Image Enhancement

Cell 5 — Resize, rotate, and flip the image

```
height, width = image.shape[:2]

resized = cv2.resize(image, (width // 2, height // 2))
center = (width // 2, height // 2)
matrix = cv2.getRotationMatrix2D(center, 30, 1.0)
rotated = cv2.warpAffine(image, matrix, (width, height))
horizontal_flip = cv2.flip(image, 1)
vertical_flip = cv2.flip(image, 0)

print("Original:", image.shape)
print("Resized:", resized.shape)
print("Rotation and flipping completed.")
```

```
Original: (H, W, 3)
Resized: (H/2, W/2, 3)  # dimensions are rounded down when odd
Rotation and flipping completed.
```

Cell 6 — Generate a negative image and crop a Region of Interest (ROI)

```
negative = 255 - image
# Example ROI: y from 50 to 200, x from 50 to 200
roi = image[50:200, 50:200]

print("Negative image generated.")
print("ROI shape:", roi.shape)

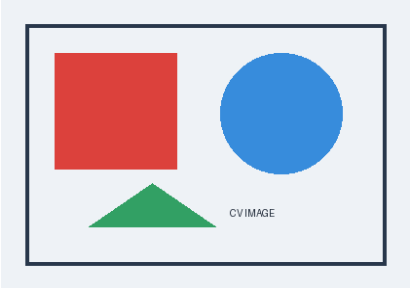
plt.figure(figsize=(10, 4))
plt.subplot(1, 3, 1); plt.imshow(image_rgb); plt.title("Original"); plt.axis("off")
plt.subplot(1, 3, 2); plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB)); plt.title("Negative"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB)); plt.title("ROI"); plt.axis("off")
plt.tight_layout()
plt.show()
```

```
Negative image generated.
ROI shape: (ROI_height, ROI_width, 3)
```

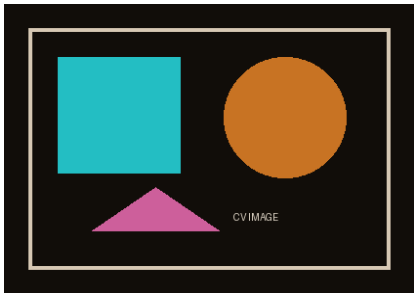
The negative operation inverts each channel value using 255 – pixel_value. An ROI extracts a selected rectangular part of the image. Change the crop coordinates to suit the image being used.

Output Comparison and Observations

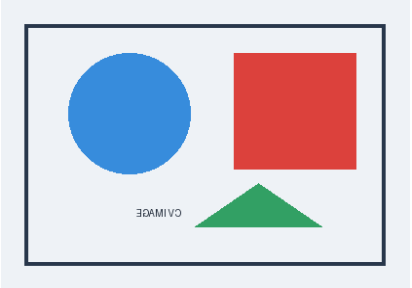
The following illustrative outputs show the general effect of common operations. The exact result depends on the input image and selected coordinates.



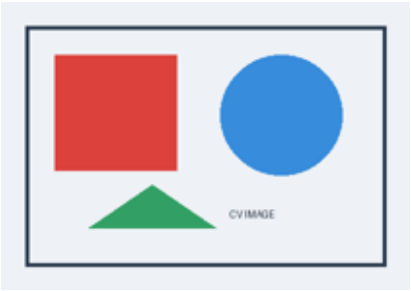
Original



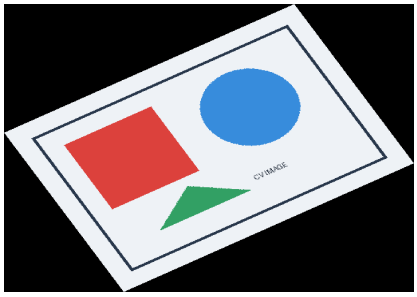
Negative



Horizontal flip



Resized



Rotated



Cropped ROI

Observations

1. Image acquisition reads image data into an array of pixel values.
2. Saving as JPEG and PNG demonstrates differences in image encoding and compression.
3. Grayscale conversion reduces three color channels to one intensity channel.
4. HSV and LAB provide alternative representations that can help with color analysis.
5. Resizing changes image dimensions; rotation changes orientation; flipping mirrors the image.
6. The negative operation reverses intensity values, while cropping extracts a Region of Interest.

Result

Fundamental image processing operations were studied and implemented using Python and OpenCV. The experiment covered image loading and saving, image-property inspection, color-space conversion, resizing, rotation, flipping, image complement, and ROI extraction. These operations provide essential preprocessing steps for feature extraction and computer vision applications.

Viva / Review Questions with Answers

1. What is a digital image? Differentiate between grayscale and color images.
A digital image is a two-dimensional array of pixels. A grayscale image stores one intensity value per pixel, while a color image commonly stores three channel values per pixel.

2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.
RGB represents red, green, and blue channels. BGR uses the same channels in a different order and is the default color order used by OpenCV when reading images. HSV represents hue, saturation, and value. LAB represents lightness (L) and two color-opponent components (A and B).

3. What is the purpose of converting an image to grayscale before further processing?
Grayscale conversion reduces the image to one intensity channel, simplifying processing and reducing the amount of data for tasks where color is not required.

4. Explain image complement (negative image) and mention its applications.
A negative image is obtained by subtracting each pixel intensity from the maximum intensity (255 for an 8-bit image). It can help highlight details in some images and is used in image enhancement and visualization.

5. Differentiate between resizing, cropping, and scaling.
Resizing changes the image dimensions. Cropping extracts a selected region without necessarily changing the pixel size of that region. Scaling enlarges or reduces image dimensions by a chosen factor.

6. What is a Region of Interest (ROI), and why is it important?
An ROI is a selected portion of an image. Processing only the ROI can focus computation on a relevant object or area and ignore unrelated background information.

7. Why is OpenCV preferred for computer vision applications?
OpenCV provides optimized image-processing and computer-vision functions, supports many image and video formats, and integrates with Python and NumPy.

8. How do image resolution and pixel intensity influence image quality?
Resolution determines the number of pixels used to represent an image and affects visible detail. Pixel intensity represents brightness; intensity range and contrast influence how clearly details can be distinguished.

9. Mention five real-world applications where image preprocessing is essential.
Medical-image analysis, autonomous driving, face recognition, surveillance, and industrial quality inspection.

10. How do preprocessing techniques improve feature extraction and deep-learning models?
Preprocessing can standardize image size, simplify color information, isolate relevant regions, and improve consistency. This can make useful features easier to detect and can help models train more reliably.

End of Experiment 1